

Rescate del workflow roto

PF1821 · Construcción de Agentes y Automatización con Herramientas Low Code
Módulo 2 · MA04560 · FUNDAMENTOS DE N8N Y MANIPULACIÓN DE DATOS · 18 h

TÉCNICA

Análisis de caso + gamificación

ORGANIZACIÓN

Individual; depuración en grupo y coevaluación en parejas

INDICADORES DE LOGRO

1.1, 3.3, 4.1, 4.2, 4.3

APOYOS

Video interactivo "Expresiones y depuración"

PRODUCTO

Workflow reparado + bitácora de depuración

SE EVALÚA CON

Rúbrica de revisión de workflow (observación) (M2-Instrumento-1.pdf); Bitácora de registro de resultados y plan de trabajo (M2-Bitacora.pdf)

APRENDIZAJES ESPERADOS

AE1. DISTINGUIR LOS CONCEPTOS FUNDAMENTALES DE AUTOMATIZACIÓN DE WORKFLOWS Y LAS CARACTERÍSTICAS DE N8N, DE ACUERDO CON CASOS DE USO EMPRESARIALES REALES.

AE3. MANIPULAR DATOS UTILIZANDO NODOS FUNDAMENTALES DE N8N, PARA TRANSFORMAR INFORMACIÓN SEGÚN REQUERIMIENTOS ESPECÍFICOS DEL WORKFLOW.

AE4. IMPLEMENTAR LÓGICA CONDICIONAL Y ESTRUCTURAS DE CONTROL EN WORKFLOWS, APLICANDO BUENAS PRÁCTICAS DE DEBUGGING Y TRAZABILIDAD.

El **insumo** `pedidos_enrutados_v0.json` lo genera el carril automático junto con `pedidos_enrutados_corregido.json`, la versión correcta para el tutor. Los dos salen de la misma definición y difieren solo en las cinco fallas de la tabla siguiente. Al montarlo, el tutor lo importa en la instancia del curso y elige las credenciales de Supabase.

Misión	Síntoma	Cómo se detecta	Causa	Corrección	Ind.
1	El pedido de \$219.000 cae en "bodega RM"	Abrir la ejecución y mirar la regla de mayorista del Switch: compara <code>total_texto</code> , que vale <code>"219.000"</code>	La regla usa <code>total_texto</code> , un texto con punto de miles armado para mostrar. Como el Switch convierte tipos, lo lee como el número 219, que no llega a 150000	Comparar <code>total</code> (Number) en la regla; el texto con formato queda solo para mostrar	4.2

Misión	Síntoma	Cómo se detecta	Causa	Corrección	Ind.
2	Todo sale por "datos inválidos"	En el nodo If, la vista previa de la expresión muestra <code>undefined</code>	La condición usa <code>{{ \$json.Email }}</code> con mayúscula; el campo se llama <code>email</code>	Cambiar a <code>{{ \$json.email }}</code> y comprobarlo con el pedido fijado	3.3
3	El pedido "distribuidor" no aparece en ninguna tabla ni da error	Historial de ejecuciones: la ejecución termina en verde pero el Switch no entrega ningún ítem	El Switch no tiene salida de respaldo: lo que no calza con ninguna regla se descarta en silencio	En <i>Options</i> del Switch, <i>Fallback Output</i> = <i>Extra Output</i> y conectarla a "revisión manual"	4.1
4	Cada pedido aparece repetido tantas veces como comunas hay	Contar los ítems de salida del Merge: 6 pedidos × N comunas	El Merge está en modo <i>Append</i> (o <i>All Possible Combinations</i>) en vez de unir por campo	Modo <i>Combine</i> → <i>Matching Fields</i> , campo <code>comuna</code> en ambas entradas	3.1
5	Supabase rechaza el pedido con <code>cantidad "3 unidades"</code> y todo se detiene	Ejecución en rojo; el error del nodo dice <code>invalid input syntax for type integer</code>	<code>cantidad</code> llega como texto que no es un número y el nodo no tiene manejo de errores	En Edit Fields, <code>cantidad</code> (Number) = <code>{{ parseInt(\$json.cantidad, 10) }}</code> ; en <i>Settings</i> del nodo Supabase, <i>On Error</i> = <i>Continue (using error output)</i> , y esa salida a <code>pedidos_rechazados</code> con el mensaje de error	4.3
Bonus	No se sabe por qué ruta pasó cada pedido	Mirar la tabla <code>pedidos</code>	Falta registrar la decisión	En cada rama, Edit Fields agrega <code>ruta</code> ("mayorista", "regiones", "bodega RM", "revisión") y <code>id_ejecucion</code> = <code>{{ \$execution.id }}</code>	4.3

Reglas del Switch en la versión corregida (modo *Rules*, se envía a la primera regla que se cumple):

Salida	Regla
0 · mayorista	<code>tipo_cliente</code> (String) <i>is equal to</i> "mayorista" OR <code>total</code> (Number) <i>is greater than or equal to</i> 150000
1 · regiones	<code>zona</code> (String) <i>is equal to</i> "Regiones"
2 · bodega RM	<code>tipo_cliente</code> (String) <i>is equal to</i> "minorista"
respaldo · revisión manual	<i>Fallback Output: Extra Output</i>

Ejemplo de una entrada de bitácora de nivel logrado (misión 3):

Síntoma: el pedido de prueba con `tipo_cliente` "distribuidor" no quedó en ninguna tabla y la ejecución salió en verde. *Detección:* en *Executions* abrí la ejecución y vi que el Switch recibió 6 ítems y entregó 5. *Causa:* ninguna regla acepta "distribuidor" y el Switch no tenía salida de respaldo, así que lo descartó sin avisar. *Corrección:* activé *Fallback Output* → *Extra Output* y la conecté a una rama que registra el pedido en `revision_manual`. *Prueba:* volví a ejecutar con los 6 pedidos fijados: 6 entran y 6 salen. *Aprendizaje:* un workflow en verde no significa que no perdió datos; hay que contar ítems de entrada y salida.

Qué distingue una respuesta débil: corrige sin explicar la causa, o "arregla" la misión 3 agregando "distribuidor" a la regla de minoristas: resuelve el caso de prueba pero el próximo valor inesperado se vuelve a perder. El indicador 4.1 pide la ruta de respaldo.